Data Structures (ETCCDS202)

Unit–2

Assignment: Linear Data Structures

Course Details

Name of the School: School of Engineering & Technology

Program: B.Tech (CSE-AI/ML)

Semester: 1 Batch: 2025-29

Course Title: Data Structures

Course Code: ETCCDS202

Student Name: Payal

Roll No.: 2501730224

Section: A

Data Structures Unit-2 Assignment Report

Question 1: Explain the amortized time complexity of append() operation in a Dynamic Array.

The append() operation in a dynamic array generally takes constant time, also known as O(1) time complexity. This means that in most cases, adding an element to the end of the array is very fast and does not depend on the number of elements already present. However, when the array becomes full, a resizing operation is required. During resizing, a new array of larger capacity (usually double the current size) is created, and all existing elements are copied into this new array. This copying process takes O(n) time, where n is the number of elements.

Even though resizing is expensive, it does not happen frequently. After resizing, many future append operations can again be done in constant time until the array fills up again. When we consider a large number of append operations together, the total cost gets distributed across all operations. This average cost per operation is called amortized time complexity.

Therefore, although some individual append operations may take O(n) time due to resizing, the overall amortized time complexity of append() remains O(1).

Question 2: Explain the time complexity of Stack and Queue operations using Linked List.

Stack and Queue are abstract data types that can be efficiently implemented using a linked list.

For Stack (LIFO - Last In First Out):
• push(x): This operation inserts an element at the beginning (head) of the linked list. Since no traversal is needed, it takes O(1) time.
• pop(): This operation removes the element from the head. It also takes O(1) time because it directly accesses the first node.
• peek(): This operation returns the top element without removing it, which is also O(1).

For Queue (FIFO - First In First Out):
• enqueue(x): This operation inserts an element at the end (tail). With a maintained tail pointer, it takes O(1) time.
• dequeue(): This operation removes the element from the front (head), which is O(1).
• front(): This operation returns the front element without removing it, also O(1).

Using linked lists ensures that all these operations are efficient because no shifting of elements is required, unlike arrays.

Question 3: Explain how a Balanced Parentheses Checker works using Stack.

A Balanced Parentheses Checker is used to determine whether an expression has properly matched and ordered brackets. It uses the Stack data structure because of its Last In First Out (LIFO) behavior.

The algorithm works as follows:
1. Traverse the expression from left to right.
2. Whenever an opening bracket (such as '(', '{', '[') is encountered, push it onto the stack.
3. Whenever a closing bracket (such as ')', '}', ']') is encountered, check the top of the stack:
   - If the stack is empty, the expression is not balanced.
   - Otherwise, pop the top element and check if it matches the corresponding opening bracket.
4. Continue this process until the entire expression is processed.
5. After traversal, if the stack is empty, the expression is balanced; otherwise, it is not.

This method ensures that every opening bracket has a matching closing bracket in the correct order. The overall time complexity of this algorithm is O(n), where n is the length of the expression.